Scenario 04 — SEUXDR CRA remediation backlog

The pilot's operative output. Twenty-five findings, ranked by how much each one blocks an EU Declaration of Conformity for **SEUXDR** (Annex III important product with digital elements, Class I). Every item is anchored to a file and line in `t4.4_siem_ai_remediation` @ `00a95ad`, and to the CRA objective and conformity question it moves.

> **Sprint 1 is closed.** Commits `c4ba16b` and `cbd092b` on branch `cra/remediation-sprint-1` (30 July 2026, 60 files, 8,261 insertions) closed BLOCK-1 through BLOCK-8 and HIGH-1, HIGH-3, HIGH-4, HIGH-8, HIGH-9, MED-1 and MED-6, taking SEUXDR from 2 of 30 CRA objectives compliant to 21 of 30. BLOCK-0 (notified body) and the items listed under *Sprint 2* in `PILOT_TIMELINE.md` remain open. The rankings below are preserved as the pilot recorded them, so the diff between backlog and outcome stays auditable.

Baseline established **2026-07-29**: 2 objectives compliant, 10 partially, 18 not compliant · conformity assessment 2 Yes / 19 Partially / 31 No · **DoC readiness 7% — Not Yet Ready** (tenant-aggregated; 2 of 30 objectives compliant for SEUXDR alone, assessment unstarted).

Assessment scope caveat. The repository ships the *authentication-stripped* variant (`CyberGuard_Stripped_Documentation.md:889`, "Stripped login and user authentication, stripped RBAC", 2025-10-29). BLOCK-1 and BLOCK-2 are therefore properties of *this build*, not of the full SEUXDR product. The first decision the backlog forces is which build is the one Clone Systems intends to place on the market.

Tier 0 — Blockers. A DoC cannot be signed while these stand.

BLOCK-0 — Decide the conformity assessment route

Annex III Class I applies on three independent limbs (SIEM system, IDS/IPS, malware removal/quarantine). **Article 32(2) does not permit a Module A internal-control self-declaration.** Engage a notified body, or commit to full application of harmonised standards once available. Nothing else in this backlog can be closed out without this decision, because it determines what "evidence" has to look like.

Owner: management · Objective: Chapter III · Question: —

BLOCK-1 — Restore authentication and authorisation

No auth middleware exists (`manager/middlewares/middleware.go`, 13 lines). No login route in `manager/routes/routes.go`. No `users` / `roles` / `sessions` tables in any of the five migrations. All 22 TLS routes are open, including org creation, agent generation, installer download (which embeds private keys) and full alert content.

The machinery to restore is already present and merely disconnected: JWT generation and validation at `manager/config/auth_config.go:102–153` (RS256, correctly rejects non-RSA `alg`), the password-

policy validator at `manager/validators/validators.go:36–63`, RS256 keys generated by `gen-certs.sh:73–81` and configured at `manager/manager.yaml:170–172`. All have **zero call sites**.

Fix: wire an authentication middleware onto the `/api` group; restore the user/role/session schema; re-point the front end's existing `LOGIN_URI` (`manager_front/src/utils/uris.ts:27`) at a real handler. Move the token out of `localStorage` (`manager_front/src/stores/authStore.ts:21–43`) into an `HttpOnly` `Secure` `SameSite` cookie.

Objective: Annex I 3b · Questions: 20, 21, 22

BLOCK-2 — Gate the remediation endpoint and record operator identity

`POST /api/agent/:agentId/execute-action` (`manager/routes/routes.go:86`) validates the action verb (`manager/handlers/agent_actions.go:385–388`) and nothing else before dispatching a root-privileged destructive action at `:454`. It is unauthenticated, unrate-limited (the limiter covers only `/api/register`, `manager/middlewares/ratelimiter.go:14–16`), unattributed (`:424–435` records no principal), and it **bypasses** `action_mode: manual_only` because the handler never reads `ACTION_MODE`.

Fix: require an authenticated, authorised operator; record principal and source IP on the command row; rate-limit the route; make `manual_only` a startup-immutable gate honoured by *every* dispatch path — including `retryCommandsForAgent` (`manager/api/messageprocessor/message_processor.go:1377–1440`), which does not read it either.

Objective: Annex I 3b, 2, 3d · Questions: 13, 20, 26

BLOCK-3 — Validate remediation targets at both ends

The only check on an LLM-derived target is non-emptiness (`manager/api/socanalystservice/react_service.go:99–101`; `agent/monitoring/monitoring.go:463–467`). No IP validation, no path canonicalisation, no protected-path or protected-process denylist exists anywhere in the repository. Prompt injection is unmitigated — attacker-controlled log text is interpolated raw via `text/template` at `manager/api/socanalystservice/prompt_service.go:206`.

Fix: validate by action type before dispatch and again before execution — strict IP regex for `BLOCK_IP`; canonicalise the path and check it against an allowlist plus a protected-path denylist for `DELETE_FILE`; validate PID or exact process name against a protected-process denylist for `KILL_PROCESS`. Strip control characters and cap length. Change `manager/config/config.go:384–386` so an unset `ACTION_MODE` defaults to `manual_only`, not `automatic`.

Objective: Annex I 1, 3d, 3i · Questions: 12, 25, 26, 33

BLOCK-4 — Publish a vulnerability disclosure policy and a security contact

No `SECURITY.md`. `readme.md:394` contains the literal empty placeholder `contact [] or open an issue`. `CyberGuard_Stripped_Documentation.md:893` points reporters at `github.com/SecureEU/seuxdr/issues` — **not this product's repository**.

Fix: publish `SECURITY.md` and a `security.txt` with a monitored contact, acknowledgement and remediation windows by severity, a CVD process, and the **Article 14 ENISA 24-hour / 72-hour reporting procedure**. This is the cheapest blocker on the list — a day of work that moves two objectives.

Objective: VH-5, VH-6 · Questions: 44, 45, 46

BLOCK-5 — Ship an SBOM

No CycloneDX or SPDX artefact exists. `licenses/credits.txt` still contains the template placeholder `<Include your full license text here>` (`:5`), declares MIT while `readme.md:390` points at Apache-2.0, and omits nearly all ~100 indirect Go modules and every front-end dependency.

Fix: generate CycloneDX for Go and npm in CI, publish per release, archive historical SBOMs against release versions, and resolve the licence contradiction. Then commit to the four process requirements the platform's SBOM Management page lists beyond generation: cadence, version tracking, machine-readable delivery and an update policy.

Objective: VH-1 · Questions: 38, 39, 40

BLOCK-6 — Stand up CI with security gates

No `.github/` , `.gitlab-ci.yml` , `Jenkinsfile` or `.circleci/` . No `govulncheck` , `trivy` , `grype` , `osv-scanner` , `npm audit` or Dependabot. No `CHANGELOG.md` . **No git tags at all** — eight unstructured commits on `main` .

Fix: a pipeline running build, `go test ./...` , `go vet` , `govulncheck` , `npm audit` and container scanning on every commit; tagged semver releases with a changelog; documented remediation SLAs by severity.

Objective: VH-2, VH-3 · Questions: 41, 48

BLOCK-7 — Draw up the risk assessment and technical documentation

Nine conformity questions (Q1–Q9) have no supporting artefact. There is no threat model, no attack-surface analysis and no data-flow diagram in the repository. Article 13 requires the cybersecurity risk assessment to be drawn up, documented, **kept updated during the support period**, and included in the technical documentation at market placement.

Fix: the CyberFort risk register, objective tree and Technical File pages produced by this pilot *are* the missing documentation — complete them and export. Add a STRIDE or PASTA threat model, an attack-surface map of the four listening ports, data-flow diagrams with trust boundaries, and explicit justifications for any requirement claimed not applicable.

Objective: Chapter II Art. 13 · Questions: 1–9

BLOCK-8 — Declare a support period of at least five years

No support-lifecycle, EOL or patch-timeline statement exists anywhere. The CRA requires a minimum five-year support period, and it is a mandatory element of the technical documentation and the user information.

Fix: publish a patch & support policy covering the support-period declaration, patch delivery timelines by severity, SLAs, end-of-support notification, customer notification process and emergency zero-day procedures.

Objective: Annex I 3k, VH-8 · Questions: 3, 6, 18, 19, 51, 52

Tier 1 — High. Individually remediable, each blocks a *Partially* from

becoming a *Yes*.

HIGH-1 — Sign releases; fail closed on missing checksums

`agent/agentd/update.go:225–228` returns `nil` — installing the binary — when the expected checksum is empty, and the field is commented out in two of three manager response paths (`manager/api/updateservice.go/update_service.go:107`; `manager/api/agentversionservice/agent_version_service.go:183`). **No code signing exists anywhere**: no GPG, Authenticode, notarisation or Sigstore. A SHA-256 delivered over the same channel as the binary defends against corruption, not against a compromised manager.

Also here: update polling runs every **10 seconds** (`agent/agentd/update.go:265`) while its own comment says 10 minutes; and the Windows installer runs `-ExecutionPolicy Bypass` (`windows_installer/main.go:32`).

Objective: Annex I 3k, VH-7 · Questions: 16, 47, 49

HIGH-2 — Authenticate the command channel; add replay protection

Port 8443 has **no `ClientAuth` or `ClientCAs`** (`manager/main.go:210–214`). Origin checking is disabled (`manager/handlers/handlers.go:33`). Transport authentication is `Authorization: ID <integer>` parsed by `strconv.Atoi` with no secret (`manager/api/agentauthenticationservice/authentication_service.go:263–286`) — and an **empty header returns `(0, nil)` and passes** (`:265–271` versus `:299–301`). The connection is registered — evicting the incumbent and triggering a command replay to the new socket (`manager/api/connectionmanager/connection_manager.go:201–207,224–235`) — *before* `CheckCredentials` runs.

Commands are **not signed**; the same symmetric key is used in both directions (`agent/monitoring/decrypt_message.go:12–13`), so a compromised agent key permits forging manager commands to that agent. `ActionMessage` (`manager/helpers/payloads.go:203–209`) has **no timestamp or sequence number** and no freshness check exists, so a captured frame replays verbatim.

Objective: Annex I 3b, 3d · Questions: 20, 25, 26

HIGH-3 — Encrypt data at rest; fix file and directory permissions

The group KEK RSA private key is stored **unwrapped** in the unencrypted SQLite database (`manager/api/configurationservice/configuration_service.go:130,134,142` → `groups.key_encryption_key`), beside the `agents.encryption_key` values it wraps — so the wrapping gives no protection against database theft. Private keys are written `0644` (`manager/helpers/helpers.go:314–337,137–152`; `keys.json` explicitly `0644` at `configuration_service.go:957,1016`). Storage and cert directories are created `0777` (`manager/main.go:36,53`). Endpoint log content is written `0644` (`manager/api/websocketservice/logstore.go:36,72`).

Fix: wrap the KEK with an OS keystore or passphrase-derived key; write all key material `0600` and all directories `0700`; enable SQLite encryption or move to an encrypted volume.

Objective: Annex I 3c, 3a · Questions: 14, 23, 24

HIGH-4 — Rotate the committed credential and purge it from history

`admin` / `S1lMfWxh1HB6SbKg.9fmkoq.P5W+07M5` is committed in three tracked files: `main.go:176–177`, `manager/api/stixservice/stix_service_integration_test.go:167–168`, `manager/api/opensearchservice/opensearch_service_integration_test.go:256–257`. Separately, the organisation API key and group licence key are **logged in cleartext** on any malformed registration (`manager/handlers/agent_actions.go:62`).

Fix: rotate, then rewrite history — a rotation alone leaves the secret recoverable by anyone who has cloned. Remove `api_key` and `license_key` from the log fields.

Objective: Annex I 3a, 3c, 2 · Questions: 13, 14, 23

HIGH-5 — Add rollback for every remediation; fix macOS blocking

`UNBLOCK_IP` is implemented agent-side (`agent/monitoring/monitoring.go:359–387`) but **no manager path can emit it** — absent from the LLM action map, from the advertised action list, and explicitly rejected by the operator endpoint. `DELETE_FILE` is an unrecoverable `rm -f` with no copy-aside, no quarantine on the live path and no restore endpoint, while the documentation claims "Delete, quarantine, or restore files". Blocks never expire: `manager/ACTIVE_RESPONSE.md:52` documents a `duration` parameter that **does not exist** in any payload struct.

On macOS the behaviour is worse than missing — `agent/active-response/seuxdr-firewall-macos.sh:44` uses `pfctl -a seuxdr -f -`, which **replaces the whole anchor ruleset**, so each new block erases the previous one while every block is recorded `completed`. `UNBLOCK_IP` flushes all rules (`:60`) and then reports success for one IP (`:65`). Enforcement state and audit trail diverge.

Fix: expose `UNBLOCK_IP` and a restore action in the manager and UI; quarantine by copy-aside instead of deleting; add TTL-based auto-revert; rewrite the macOS script to use `pfctl -t <table> -T add/delete`.

Objective: Annex I 3g, 3i · Questions: 31, 33

HIGH-6 — Move safety controls onto the live execution path

The whitelist (`agent/helpers/helpers.go:523–549`), dangerous-pattern blocklist (`:578–583`) and execution timeout (`:467–472`) live only in `ExecuteActiveResponseCommand`, reachable only from `command`-type messages — **which the manager never sends**. The live `action` path (`agent/monitoring/monitoring.go:446–484`) has three empty-string checks and **no timeout at all** (`exec.Command`, not `CommandContext`).

Even where the whitelist is reachable it is defeatable: `buildPowerShellCommand` (`helpers.go:603–607`) flattens the command and all arguments into one `powershell -Command` string; the blocklist omits newline, bare `>` / `>>` / `2>&1`, `&`, `,` and `iex`; the whitelist matches **basename only** (`:552–560`), so `/tmp/evil/mv` passes; and `Environment` is appended to `os.Environ()` unvalidated (`:497–499`), permitting `LD_PRELOAD` and `PATH` injection.

Fix: move validation, whitelisting and timeout enforcement into `MonitoringService.ExecuteAction`, or retire the action path in favour of the validated command path. Pass arguments as argv, never through `-Command`.

Objective: Annex I 3i, 3d · Questions: 25, 33

HIGH-7 — Reduce container attack surface

`docker-compose.yml:11` sets `privileged: true` on the manager — effectively no container boundary. `Dockerfile:46` appends `root ALL=(ALL) NOPASSWD:ALL` to sudoers and `:48-49` deliberately weakens PAM. `docker-compose.yml:39-40` **publishes Ollama's port 11434 to the host** despite an internal network being defined, and Ollama has no authentication. `manager/start-server.sh:15-18` runs `go mod download` and `go run main.go` **inside the running container** as root — there is no immutable compiled artefact and the dependency tree is refetched from the network on every start. `Dockerfile:53` and `startup.sh:19` fetch and execute the Wazuh installer with **no checksum or signature verification**, as does the Go toolchain download (`Dockerfile:13-22`).

Objective: Annex I 3h, 3a · Questions: 14, 32

HIGH-8 — Minimise and protect endpoint log content

Raw log lines are persisted verbatim to `soc_analyst_analyses.full_log`, written to world-readable queue files, and interpolated into LLM prompts. They routinely contain usernames, hostnames, source IPs, file paths and command lines — personal data under GDPR. There is no pseudonymisation, no field redaction, and **no retention policy at all** on `soc_analyst_analyses` or `active_response_commands`; both grow without bound.

Fix: redact or pseudonymise identifiers before prompt construction and storage; add a documented retention policy per table; tighten file modes to `0600`; publish a personal-data statement.

Objective: Annex I 3e, 3c · Questions: 23, 28, 36

HIGH-9 — Establish a dependency policy and update the stale tree

Direct dependencies pinned to unreleased or pre-module versions: `golang-migrate v3.5.4+incompatible` (2018), **`gorilla/websocket v1.4.2` (2020 — this carries the command channel)**, `patrickmn/go-cache v2.1.0+incompatible` (2019), `kardianos/service` and `tkanos/gonfig` (2021 commits, no tagged release), `golang/mock` and `pkg/errors` (archived upstream). Duplicated stacks: two SQLite drivers, two mock libraries, and **three faker libraries in production front-end dependencies** including the sabotaged `faker@6.6.6`. No container image is digest-pinned; `redhat/ubi9` has no tag and `ollama/ollama` floats on `latest`. `manager_front/Dockerfile:11` uses `npm install --frozen-lockfile` — a Yarn flag that is a **no-op for npm**, so the lockfile is not enforced; use `npm ci`.

Objective: VH-1, VH-2, Annex I 2 · Questions: 13, 38, 48

Tier 2 — Medium. Credibility and audit-trail integrity.

MED-1 — Reconcile documentation with code, or delete the claims

For a regulatory submission this is the most dangerous category on the list, because it invites an assessor to accept assurance the artefact does not provide — and it undermines the eight genuine conformity claims.

Document	Claim	Reality
`CLAUDE.md:18-19,45`	RBAC, JWT authentication, fine-grained permissions	none exist in this build
`CLAUDE.md:220,243,245`	`sessions` table, all sensitive data encrypted, rate limiting on all endpoints	no such table; DB unencrypted; limiter covers one route
`setup-docker.sh:1019,1027`	"Default admin credentials are created on first run"	**no such code exists**
`manager/ACTIVE_RESPONSE.md:110-111`	agent authentication and role-based permissions for command execution	neither exists
`ACTIVE_RESPONSE_IMPLEMENTATION.md:26-29`	distro-aware command generation (`iptables` vs `firewall-cmd`)	no distro branching; the code is dead anyway
`ACTIVE_RESPONSE_IMPLEMENTATION.md:83`	`>` and `2>` are blocked	only the backtick-suffixed variants are; bare redirection is not
`CyberGuard_Stripped_Documentation.md:141-142`	host isolation; file quarantine and restore	isolation does not exist; quarantine has no agent handler
`README_TESTING.md`	confidence-threshold behaviour	`confidence` has zero non-test hits in the codebase

Objective: Chapter II Art. 13, Art. 25 · Questions: 8, 10

MED-2 — Implement the inert safety knobs, or remove them

Each is configured, validated at load, documented — and never enforced: `min_rule_level` (used only to build a prompt string), `max_alerts_batch` (query hard-codes `"size": 10000`), `cooldown_period` (no timestamp comparison anywhere), `command_timeout` (hard-coded 30 on the live path), `confidence_thresholds` (**not in the config struct at all**), `analysis_timeout` (overridden by a hard-coded 60 s), `llm.temperature` and `max_tokens` (silently ignored for Ollama), `fallback_to_rules` (plumbed and never read).

Net effect: **every alert of any severity that resolves to a connected agent goes to the LLM and may yield up to three host actions, with no cooldown, no batch cap and no confidence gate.**

Objective: Annex I 1, 3a · Questions: 12, 14

MED-3 — Make the audit trail attributable, complete and durable

No operator identity on any command row. `soc_analysis_id` and `soc_action_id` are never written, so the lineage columns are permanently NULL. `soc_analyst_analyses.alert_id` and `processing_time_ms` are never written — an AI decision cannot be traced to its originating alert. **The prompt, model name, model digest, temperature and token counts are persisted nowhere, so an AI remediation decision is not reproducible.** `ON DELETE CASCADE` on `agent_id` (`000004_soc_analyst_storage.up.sql:21,38`) **destroys all AI decision history when an agent is**

deleted. No hash chain or signature, so no tamper-evidence. The agent-side result record is **deleted on successful upload** (`agent/db/active_response_results_repository.go:38-40`). `manager/api/activeresponseservice/active_response_service.go:530-535` still carries `// TODO: Store command result in database for audit trail`.

Objective: Annex I 3j · Questions: 27, 34, 35

MED-4 — Eliminate duplicate execution

`retryCommandsForAgent` fires on **every** `RegisterConnection` and re-sends every command still `pending` or `connection_failed`. Commands are created `pending` (`message_processor.go:1261`) and **never advanced to `sent`** on success, so a flapping agent re-executes the same `DELETE_FILE` or `KILL_PROCESS` on every reconnect until the ~60 s expiry sweep. Retries also re-run the full LLM analysis up to three times, including after a *partial* success.

Fix: advance status to `sent` on dispatch; add an idempotency key per `(alert_id, agent, action, target)`.

Objective: Annex I 3g, 3i · Questions: 31, 33

MED-5 — Fix the development PKI story

`gen-certs.sh:15-18` mints a **1000-day RSA-2048** CA with `CN=testServerCA`, and `readme.md:203-223` instructs operators to import it as a **trusted root** in the browser while `import.sh:2` adds it to the macOS System keychain as `trustRoot`. Serial numbers are hard-coded constants `01` and `02`, breaking X.509 serial uniqueness. No `-sha256` is specified on the signing operations. `localhost.ext:8` and `manager/manager.yaml:4,109,129,149` hard-code `192.168.0.154`.

Note the contrast worth preserving: the **runtime** mTLS PKI is materially stronger — RSA-4096, 10-year CA, 1-month client certificates with 7-day refresh and automated re-signing.

Objective: Annex I 3a, 3c · Questions: 14, 24

MED-6 — Establish a single product version identifier

The agent declares `1.0.1`, the front end declares `0.0.0`, and **the manager has no version identifier anywhere** — no field in `manager.yaml`, no build stamp, no git tag. `agent_versions.release_notes` is seeded with the placeholder `'Bug fixes and improvements'`. *Product Identification* is a mandatory DoC section and cannot be completed without this.

The product also carries **three overlapping names** — `SEUXDR` (the Go module), `CyberGuard` (the documentation and logo) and `SECUR-EU SIEM` / `NEXT GEN SIEM` (the shipped UI). Pick one for the declaration.

Objective: Chapter II Art. 13 · Questions: 7, 8

Tier 3 — Low. Hygiene, but an assessor will notice.

- **LOW-1 — Remove the committed 35 MB unsigned binary.** `manager/manager` is a tracked Mach-O arm64 executable with unverifiable provenance, shipped to every clone. Also committed: `manager/db/test.html` (an iframe pointing at a hard-coded internal host) and `agent/test_file.txt`. *Add to `.gitignore` and purge from history.*

- **LOW-2 — Delete dead code that inflates the apparent posture.** File integrity monitoring (`agent/monitoring/integrity.go:11–70`, zero call sites) is documented as a feature but is not operative. Rootkit detection appears in agent config with no implementation. Windows registry monitoring is absent entirely. Thirteen OSSEC-legacy response scripts are embedded in every agent binary with zero code references. A **second complete agent executor** at `agent/helpers/helpers.go:685–967` is used only by tests. A reviewer reading these files will reach wrong conclusions — as will an assessor.
- **LOW-3 — Fix the temp-file execution race.** `agent/monitoring/monitoring.go:265–323` writes the response script to a **predictable** `%TEMP%\seuxdr_<action>_<UnixNano>.cmd`, chmods `0755` and executes it. `%TEMP%` is frequently writable by unprivileged users in service contexts, giving a TOCTOU replacement window on a script the SYSTEM-privileged agent is about to run. `/tmp/seuxdr_logs` is created world-traversable at `0755`.
- **LOW-4 — Quote script variables.** `agent/active-response/win/kill-process.cmd:31` expands `%PROCESS%` unquoted into a `FINDSTR` pipeline; `win/firewall-drop.cmd:33` uses unquoted `remoteip=%SRCIP%`; `firewall-drop.sh:142–146` word-splits `${ARG1}`, so a target like `1.2.3.4 -j ACCEPT` becomes iptables argument injection; `seuxdr-firewall-macos.sh:44` pipes the target through `echo` into `pfctl -f -`, so a newline injects arbitrary pf rules.
- **LOW-5 — Guard the agent's WebSocket listener.** `go monitoringSvc.ListenWSAR()` is spawned from inside the reconnect branch (`agent/monitoring/loglisten.go:45`) with no guard and no cancellation of the previous goroutine, so each reconnect adds another reader on the shared connection. `gorilla/websocket` does not support concurrent readers.
- **LOW-6 — Clean up the build.** `Dockerfile:8–22` downloads Go 1.22.5 then discards it for 1.24.2 while `go.mod:3` declares 1.23 — three versions in play, two toolchain downloads per build. `agent/config/agent_base_config.yml:3–4` ships stale port defaults. `manager/database/migrations/000001_init_mg.down.sql` drops `organizations` twice (the table is `organisations`) and drops a non-existent `users` table, so the down-migration is broken.
- **LOW-7 — Correct the ReAct loop or rename it.** The "Observation" step is fabricated: no tool is executed and the observation is the synthetic string `"Action recorded: %s %s"` (`manager/api/socanalystservice/react_service.go:227`). It is iterated generation without grounding feedback, not ReAct. Either execute read-only tools to ground it, or describe it accurately.

Sequencing

Week 1 — decide and declare. BLOCK-0 (assessment route) and BLOCK-4 (`SECURITY.md`) are the cheapest, highest-leverage moves. BLOCK-5 (SBOM) and BLOCK-6 (CI) are largely tooling. MED-1 (reconcile docs) costs nothing but attention and protects everything else.

Weeks 2–4 — close the access-control blockers. BLOCK-1, BLOCK-2, BLOCK-3, then HIGH-1, HIGH-2, HIGH-3, HIGH-4. These are the items that make Q13 and Q20 answerable as anything other than *No*.

Weeks 5–8 — documentation and process. BLOCK-7, BLOCK-8, HIGH-8, HIGH-9, MED-2, MED-3, MED-6.

Then re-run the pilot. The compliance chain is live: re-scan, re-answer and read the new DoC readiness score. That repeatability — not the first score — is the platform's actual value, and the second run is where the number should move sharply.

What good looks like

A realistic target after the Tier 0 and Tier 1 work, assuming the notified-body route is under way:

Metric	Baseline (2026-07-29)	Target
Objectives compliant	2 / 30	20 / 30
Objectives partially compliant	10 / 30	8 / 30
Objectives not compliant	18 / 30	2 / 30
Conformity answers *Yes*	2 / 52	34 / 52
Objectives with evidence attached	0 / 30	30 / 30
DoC readiness	**7% — Not Yet Ready**	**> 70%**

The two objectives that should stay *partially compliant* on purpose are `3g` and `3i` — remediation rollback and blast-radius controls are genuinely hard for an autonomous-response product, and an honest *Partially* with a documented compensating control is worth more to an assessor than an unsupported *Yes*.